# INFORME PRELIMINAR

## Implementación de Tetris mediante **Manipulación de Bits en C++**

Informática II — Ingeniería Electrónica

Fase: Análisis y Diseño de la Solución

Profesores: Aníbal Guerra

Estudiante: David Ortiz

2026

# 1. Contextualización del Problema

Este proyecto consiste en desarrollar una versión simplificada del juego Tetris que opera completamente a través de la consola. A diferencia del Tetris tradicional, la ejecución no es en tiempo real: el tablero se re-imprime en cada acción del usuario, y la gravedad actúa por turno, no por tiempo.

El desafío central radica en implementar la lógica del juego haciendo uso obligatorio de operadores a nivel de bits (bitwise) para la representación del tablero, el manejo de piezas, la detección de colisiones, la rotación en tiempo de ejecución y la eliminación de filas. Se prohíbe explícitamente el uso de la librería STL, estructuras predefinidas de C++, y el tipo string.

## 1.1 Restricciones clave del enunciado

- Implementado en C++ bajo el framework Qt. Sin ANSI C.
- Programación modular: múltiples archivos .h y .cpp.
- Uso de punteros, arreglos y memoria dinámica obligatorio.
- Sin estructuras, librerías no autorizadas ni STL.
- Sin tipo string. Sin objetos string.
- Las rotaciones de las piezas deben calcularse en tiempo de ejecución mediante máscaras de bits, no guardarse precalculadas.
- El ancho del tablero debe ser múltiplo de 8. Alto sin restricción especial (mínimo 8).
- Las piezas aparecen centradas en la parte superior en su orientación original.
- Se permite uso de librerías para generación de números aleatorios.

## 1.2 Dinámica de juego

El flujo es por turnos. Tras cada acción del usuario (A=izquierda, D=derecha, S=bajar, W=rotar, Q=salir), el sistema procesa el movimiento, verifica colisiones, fija la pieza si corresponde, elimina filas completas si hay, genera la nueva pieza, y re-imprime el tablero completo en consola. La gravedad equivale a que cada acción 'bajar' mueve la pieza un paso hacia abajo.

# 2. Análisis del Problema

## 2.1 Identificación de entidades principales

Del enunciado se extraen cuatro entidades fundamentales que deben ser modeladas:

| Entidad | Responsabilidad | Relación con bits |
|---|---|---|
| Tablero | Almacenar el estado de cada celda del juego | Arreglo de enteros; cada bit = celda ocupada/vacía |
| Pieza activa | Representar la figura en juego, su posición y rotación actual | Máscara de bits de NxN; se rota aplicando transformaciones bit |
| Cola de filas | Saber qué filas están llenas para eliminarlas | Verificación de que todos los bits de una fila estén en 1 |
| Motor de juego | Coordinar turnos, colisiones, gravedad y game over | Usa AND/OR para insertar/detectar piezas en el tablero |

## 2.2 Representación del tablero con bits

Esta es la decisión de diseño más importante. El tablero se representa como un arreglo dinámico de enteros sin signo (unsigned int o unsigned char, según el ancho elegido). Cada fila del tablero es un entero, y cada bit de ese entero corresponde a una columna: bit=1 significa celda ocupada, bit=0 significa celda vacía.

**Ejemplo de representación de una fila con ancho = 8 columnas**

```
Tipo de dato: unsigned char  (8 bits exactos = 8 columnas)
Fila vacía:   0b00000000  =  0
Fila llena:   0b11111111  =  255
Celda 0 y 7:  0b10000001  =  129

Para ancho > 8: usar unsigned short (16 bits), unsigned int (32 bits), etc.
El ancho debe ser múltiplo de 8, lo que se alinea perfectamente con los tamaños de tipos enteros.
```

La selección del tipo de dato para las filas depende del ancho ingresado por el usuario en tiempo de ejecución. Se usará un puntero a unsigned int (capaz de cubrir hasta 32 columnas). Si el ancho es mayor, se puede usar un arreglo de unsigned int por fila.

## 2.3 Representación de las piezas

Cada pieza (figura) se representa como una máscara de bits en una grilla de 4x4. Esto ocupa exactamente 16 bits, que caben en un unsigned short. Las 7 figuras clásicos en su orientación original son:

| Pieza | Patrón visual 4x4 | Valor hexadecimal (aprox.) |
|---|---|---|
| I (línea) | 0000 / 1111 / 0000 / 0000 | 0x0F00 |
| O (cuadrado) | 0000 / 0110 / 0110 / 0000 | 0x0660 |
| T | 0000 / 0100 / 1110 / 0000 | 0x04E0 |
| S | 0000 / 0110 / 1100 / 0000 | 0x06C0 |
| Z | 0000 / 1100 / 0110 / 0000 | 0x0C60 |
| J | 0000 / 1000 / 1110 / 0000 | 0x08E0 |
| L | 0000 / 0010 / 1110 / 0000 | 0x02E0 |

IMPORTANTE: Estas máscaras no son los valores definitivos de implementación; son una referencia conceptual. Los valores exactos dependerán del esquema de bits que se defina (bit más significativo = columna izquierda o derecha).

## 2.4 Rotación de piezas en tiempo de ejecución

Una restricción fundamental del enunciado es que no se pueden guardar las rotaciones precalculadas. Cada rotación debe generarse algorítmicamente a partir de la orientación actual, usando operaciones de bits.

El algoritmo de rotación 90° horaria para una grilla NxN es: la columna j de la nueva pieza = fila (N-1-j) de la original. Para una máscara de 4x4 en 16 bits, esto se traduce en operaciones de desplazamiento y enmascaramiento:

```
// Rotación 90° horaria de máscara 4x4 almacenada en unsigned short
// El bit (fila i, col j) está en la posición: i*4 + j  (si MSB = fila 0, col 0)

unsigned short rotarHorario(unsigned short mascara) {
    unsigned short resultado = 0;
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            int bitOrigen = (i * 4) + j;
            int bitDestino = (j * 4) + (3 - i);
            if (mascara & (1 << (15 - bitOrigen))) {
                resultado |= (1 << (15 - bitDestino));
            }
        }
    }
    return resultado;
}
```

La rotación se acumula: la pieza mantiene solo su máscara actual y al presionar W, se llama a rotarHorario(). No se guarda ningún array de rotaciones.

## 2.5 Colisión y fijación

Para detectar si una pieza puede moverse a una posición (x, y), se extraen los bits activos de la pieza, se desplazan a la posición correspondiente en el tablero, y se aplica AND con las filas del tablero:

```
// Pseudocódigo de detección de colisión
// piece_mask: mascara 4x4 de la pieza
// board[]: arreglo de filas del tablero (cada elemento = una fila)
// px, py: posición (col, fila) de la esquina superior izquierda de la pieza

bool hayColision(unsigned short piece_mask, int px, int py) {
    for (int i = 0; i < 4; i++) {
        // Extraer fila i de la máscara 4x4
        unsigned char fila_pieza = (piece_mask >> (12 - i*4)) & 0xF;
        // Ajustar a posición horizontal px
        unsigned int fila_desplazada = fila_pieza << (anchoTablero - 4 - px);
        // Verificar contra el tablero y contra los bordes
        if (py + i >= altoTablero) return true;  // borde inferior
        if (fila_desplazada & ~mascaraTablero) return true; // borde lateral
        if (fila_desplazada & board[py + i]) return true;   // colisión pieza
    }
    return false;
}
```

## 2.6 Eliminación de filas

Una fila está completa cuando todos sus bits están en 1. Esto se detecta comparando el valor de la fila con la máscara de fila llena (todos los bits del ancho activos):

```
// mascaraLlena = (1 << ancho) - 1   =>   todos los bits del ancho en 1
// Ejemplo ancho=8: mascaraLlena = 0xFF = 0b11111111

bool filaLlena(unsigned int fila) {
    return (fila & mascaraLlena) == mascaraLlena;
}

// Al eliminar una fila, las superiores bajan (se copian usando punteros)
// board[i] = board[i-1]  para i desde fila_eliminada hasta 1
```

```
// board[0] = 0  (nueva fila vacía en la cima)
```

# 3. Diseño de la Solución

## 3.1 Arquitectura modular del proyecto

El proyecto sigue una arquitectura multi-archivo (.h / .cpp). Cada módulo tiene una responsabilidad clara y se comunica con los demás a través de punteros y estructuras básicas:

| Archivo | Contenido / Responsabilidad |
|---|---|
| main.cpp | Punto de entrada. Inicializa el juego y llama al loop principal. |
| tablero.h / tablero.cpp | Estructura del tablero, init, impresión, detección de fila llena, eliminación. |
| pieza.h / pieza.cpp | Definición depiezas, generación aleatoria, rotación bitwise, offsets. |
| motor.h / motor.cpp | Lógica central: colisiones, movimientos, fijación, game over, gravedad. |
| input.h / input.cpp | Lectura de acciones del usuario por teclado. |
| utils.h / utils.cpp | Funciones auxiliares: imprimir bits, limpiar pantalla, etc. |

## 3.2 Estructura de datos: Tablero

El tablero es una estructura que contiene un puntero doble para el arreglo dinámico de filas, y los metadatos necesarios:

```
// tablero.h
typedef struct {
    unsigned int** filas;   // arreglo dinámico de punteros a filas
    // Alternativa simple: unsigned int* filas (arreglo 1D, índice = fila)
    int ancho;              // columnas (múltiplo de 8)
    int alto;               // filas (mínimo 8)
    unsigned int mascaraLlena; // (1 << ancho) - 1
} Tablero;

Tablero* crearTablero(int ancho, int alto);
void destruirTablero(Tablero* t);
void imprimirTablero(Tablero* t, Pieza* piezaActiva, int px, int py);
bool filaLlena(Tablero* t, int fila);
void eliminarFila(Tablero* t, int fila);
void fijarPieza(Tablero* t, Pieza* p, int px, int py);
```

## 3.3 Estructura de datos: Pieza

La pieza activa se representa con su máscara 4x4 actual (post-rotación), su tipo original y su posición en el tablero. La máscara se recalcula mediante rotación bitwise al girar, nunca se almacena en array:

```
// pieza.h
typedef struct {
    unsigned short mascara;  // estado actual 4x4 (puede estar rotada)
    int tipo;                // 0=I, 1=O, 2=T, 3=S, 4=Z, 5=J, 6=L
    int px;                  // columna de la esquina sup-izq en el tablero
    int py;                  // fila de la esquina sup-izq en el tablero
} Pieza;

// Máscaras originales (definidas en pieza.cpp)
// const unsigned short PIEZAS_ORIG[7] = { ... };

Pieza* crearPiezaAleatoria(int anchoTablero);
void rotarPieza(Pieza* p);          // rota en tiempo de ejecución
void destruirPieza(Pieza* p);
```

## 3.4 Estructura de datos: Motor

El motor no tiene estructura propia; opera sobre punteros al tablero y a la pieza activa. Sus funciones implementan la lógica central del juego:

```
// motor.h
bool puedeMoverse(Tablero* t, Pieza* p, int dx, int dy);
void moverIzquierda(Tablero* t, Pieza* p);
void moverDerecha(Tablero* t, Pieza* p);
void bajarPieza(Tablero* t, Pieza* p);  // baja 1; fija si no puede
void girarPieza(Tablero* t, Pieza* p);  // rota si no colisiona
bool esGameOver(Tablero* t, Pieza* p);  // verifica si nueva pieza cabe
void procesarFilasLlenas(Tablero* t);   // elimina todas las filas llenas
```

# 4. Algoritmos Clave

## 4.1 Ciclo principal del juego

```
// main.cpp — flujo principal
inicializar tablero (pedir ancho y alto al usuario, validar múltiplo de 8)
pieza = crearPiezaAleatoria(tablero->ancho)

mientras (!gameOver):
    imprimirTablero(tablero, pieza, pieza->px, pieza->py)
    mostrar instrucciones: [A]Izq [D]Der [S]Bajar [W]Rotar [Q]Salir
    leer accion del usuario

    si accion == 'A': moverIzquierda(tablero, pieza)
    si accion == 'D': moverDerecha(tablero, pieza)
    si accion == 'S': bajarPieza(tablero, pieza)
       → si la pieza se fija:
             fijarPieza(tablero, pieza, pieza->px, pieza->py)
             procesarFilasLlenas(tablero)
             destruirPieza(pieza)
             pieza = crearPiezaAleatoria(tablero->ancho)
             si esGameOver(tablero, pieza): gameOver = true
    si accion == 'W': girarPieza(tablero, pieza)
    si accion == 'Q': gameOver = true

imprimir 'GAME OVER'
liberar memoria: destruirTablero, destruirPieza
```

## 4.2 Impresión del tablero

Al imprimir, se combina temporalmente el tablero fijo con la pieza activa (sin modificar el tablero), extrayendo bit a bit cada celda:

```
// Para cada fila y del tablero:
//   Obtener fila_base = tablero->filas[y]
//   Si la pieza activa ocupa esta fila (py <= y < py+4):
//     Extraer fila_pieza de la máscara, desplazarla a px
//     fila_mostrar = fila_base | fila_pieza
//   Para cada bit de fila_mostrar, imprimir '#' si bit=1, '.' si bit=0
//   Imprimir borde '|' al inicio y fin de cada fila

void imprimirTablero(Tablero* t, Pieza* p, int px, int py) {
```

```
    for (int y = 0; y < t->alto; y++) {
        unsigned int fila = t->filas[y];
        // Superponer pieza activa si intersecta esta fila
        if (p != NULL && y >= py && y < py + 4) {
            int rel = y - py;
            unsigned int fila_p = (p->mascara >> (12 - rel*4)) & 0xF;
            fila_p = fila_p << (t->ancho - 4 - px);
            fila |= fila_p;
        }
        // Imprimir bit a bit
        printf("|");
        for (int x = t->ancho - 1; x >= 0; x--) {
            printf("%c", (fila >> x) & 1 ? '#' : '.');
        }
        printf("|\n");
    }
}
```

# 5. Plan de Desarrollo e Hitos (Commits)

El desarrollo se organiza en hitos incrementales, cada uno con su propio commit en el repositorio. Cada hito produce un prototipo funcional parcial:

| Hito / Commit | Descripción | Criterio de verificación |
|---|---|---|
| Hito 1:Estructura base | Creación de todos los .h con estructuras y prototipos. main.cpp con esqueleto. | Compila sin errores. Archivos organizados. |
| Hito 2:Tablero y display | Implementar crearTablero, destruirTablero, imprimirTablero. Tablero vacío funcional. | Se muestra tablero vacío con bordes correctos en consola. |
| Hito 3:Piezas y máscaras | Definir las 7 máscaras. crearPiezaAleatoria. Pieza aparece centrada arriba. | Se genera pieza aleatoria y se muestra superpuesta en el tablero. |
| Hito 4:Rotación bitwise | Implementar rotarPieza() con el algoritmo de transposición de bits. | Presionar W rota visualmente la pieza en pantalla. |
| Hito 5:Movimiento y colisión | Implementar puedeMoverse, moverIzquierda/Derecha, bajarPieza. | Pieza se mueve respetando bordes y piezas fijadas. |
| Hito 6:Fijación y filas | Implementar fijarPieza y procesarFilasLlenas. | Al llegar al fondo, pieza queda fija. Filas completas se eliminan. |
| Hito 7:Game Over y cierre | Implementar esGameOver. Liberar toda la memoria al salir. | Juego detecta Game Over. No hay memory leaks (valgrind). |
| Hito 8:Pulido final | Limpiar pantalla entre impresiones, mensajes de puntaje/nivel opcionales. | Experiencia de juego completa y fluida. |

## 5.1 Consideraciones de memoria dinámica

- El tablero (filas) se aloja con malloc/new al inicio y se libera con free/delete al salir.
- Cada Pieza activa se aloja con new al crearse y se libera con delete al fijarse.
- No se usan variables locales para estructuras grandes que requieran persistencia entre funciones.
- Se valida que malloc/new no retornen NULL antes de usar los punteros.

## 5.2 Posibles problemas anticipados

- Alineación de bits de la pieza con el tablero: el desplazamiento depende del ancho del tablero y la posición px. Requiere pruebas cuidadosas.
- Rotación que saca la pieza del borde: se debe verificar colisión antes de aceptar la rotación, y revertir si colisiona.
- Pieza I en posición vertical: ocupa 4 filas; hay que manejar el caso en que parte de la máscara 4x4 quede fuera del tablero ($y < 0$).

- Fijación de pieza cuando parte está fuera de la pantalla (spawn en la cima): se debe verificar que sea Game Over solo cuando la parte visible colisione.

## 6. Resumen y Conclusiones del Análisis

La solución propuesta aborda todos los requisitos del enunciado de manera directa y eficiente. Las decisiones de diseño más importantes se resumen a continuación:

| Decisión | Justificación |
|---|---|
| Tablero como arreglo de unsigned int* | Permite representar cada fila como un entero, habilitando operaciones bitwise eficientes. El tipo se ajusta al ancho del tablero. |
| Piezas como unsigned short (16 bits = grilla 4x4) | Compacto, suficiente para todas las figuras, y permite algoritmos de rotación bit a bit. |
| Rotación por transposición de bits en tiempo de ejecución | Cumple la restricción del enunciado. El algoritmo es O(16) operaciones, completamente viable. |
| Memoria dinámica para tablero y piezas | Cumple requisito. El tamaño del tablero no se conoce en compilación (lo ingresa el usuario). |
| Sin STL ni string | Se usan arreglos C puros, printf/scanf, y punteros. Compatible con las restricciones del curso. |

Este informe preliminar constituye la base conceptual y de diseño sobre la cual se desarrollarán los commits sucesivos del repositorio. Cada hito implementará una capa funcional adicional, verificable de forma independiente.